Rapport BDA (Projet 1)

Espaces verts

- **Liste des dépendances fonctionnelles non-triviales trouvés.**

nsq_espace_vert -> nom_ev
nsq_espace_vert -> type_ev
nsq_espace_vert -> categorie
nsp_espace_vert -> adresse_numero, adresse_typevoie, adresse_libellevoie
nsq_espace_vert -> poly_area
nsq_espace_vert -> surface_totale_reelle
nsq_espace_vert -> surface_horticole
nsq_espace_vert -> perimeter
nsq_espace_vert -> annee_ouverture
nsq_espace_vert -> annee_renovation
nsq_espace_vert -> ancien_nom_ev
nsq_espace_vert -> annee_changement
nsq_espace_vert -> presence_cloture
nsq_espace_vert -> ouvert_ferme
nsq_espace_vert -> adresse_complement
nsq_espace_vert -> sites_villes
nsq_espace_vert -> id_atelier_horticole
nsq_espace_vert -> id_division
nsq_espace_vert -> competence
nsq_espace_vert -> id_eqpt
nsq_espace_vert -> geom_x_y
nsq_espace_vert -> last_edited_user
nsq_espace_vert -> last_edited_date

(Je n'ai pas compris à quoi correspondent ces données donc je les ai laissé avec la clé principale)
nsq_espace_vert -> nb_entite
nsq_espace_vert -> ida3d_enb

(Attention il y a 2 NULL pour geom_x_y et geom, peut être une erreur?)
geom_x_y -> geom
geom_x_y -> url_plan

(Dans le cas où 0, NULL et XXXX sont censés représenter l'absence de données on peut créer une table Equipement avec NOT NULL sur l'id_eqpt)
id_eqpt -> id_division

(Si on oblige les informations d'adresse à être non NULLsauf le complément)
adresse_numero, adresse_typevoie, adresse_libellevoie -> adresse_codepostal

On pourrait également regrouper les informations d’adresses en adresse_complete pour nous faciliter la tâche.

- Quelques requêtes que vous développerez pour vérifier que les dépendances fonctionnelles sont effectivement vérifiées dans les données. Concentrez vous sur les cas non-triviaux.
Voir ev_query.sql

- Une représentation graphique d'une couverture minimale de dépendances fonctionnelles. (Vous pouvez utiliser graphviz par exemple.)
Voir espaces_verts.pdf ou espaces.png

- Une décomposition en FNBC avec une justification (ne soyez pas exhaustif, concentrez vous sur les cas que vous jugez intéressants et qui méritent une explication).
espaces_verts: (nsq_espace_vert, nom_ev, type_ev, categorie, poly_area, surface_totale_reelle, surface_horticolore, permettre, annee_ouverture, annee_renovation, ancien_nom_ev, annee_changement, presence_cloture, ouvert_ferme, id_atelier_horticolore, competence, last_edited_user, last_edited_date)

equipement(id_eqpt, id_division) pour éviter une dépendance transitive nsq_espace_vert -> id_eqpt -> id_division alors que id_eqpt n'est pas une clé.

geo(geom_x_y, geom, url_plan) il aurait fallu séparer geom_x_y et geom car on risque geom_x_y -> geom -> url_plan et geom -> geom_x_y -> url_plan ou peut être laissé url_plan dans espaces_verts si elle ne dépend pas des deux geom.

adresse(adresse_numero, adresse_typevoie, adresse_libellevoie, adresse_complement, adresse_codepostal) on n'insère plus d'adresse incomplète et on sépare pour ne pas avoir de transitivité non plus.

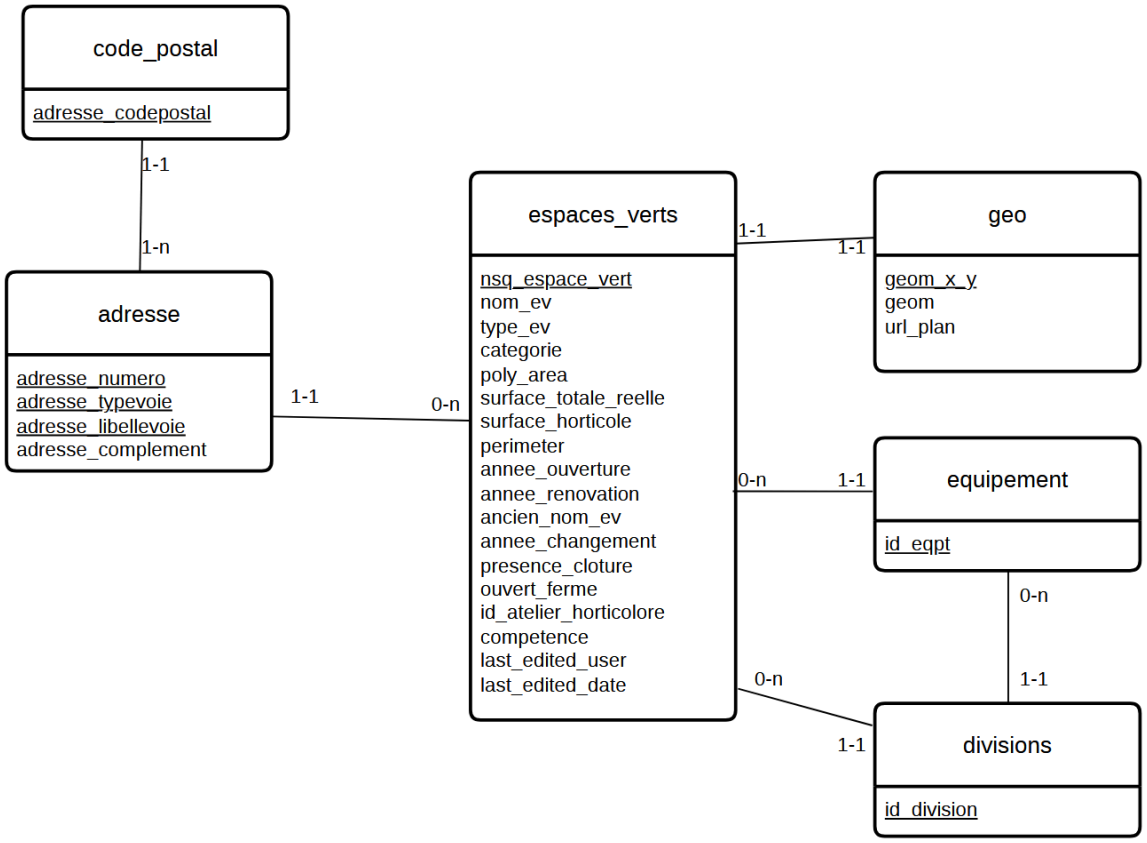
- Pour chaque table de la décomposition, l'ensemble des clefs candidates, et les dépendances fonctionnelles locales.
Clés candidates:

- nsq_espace_vert
- geom
- geom_x_y
- adresse_numero, adresse_typevoie, adresse_libellevoie
- adresse_codepostal
- id_division

Dépendances fonctionnelles:

- nsq_espace_vert -> nom_ev, type_ev, categorie, poly_area, surface_totale_reelle, surface_horticolore, permettre, annee_ouverture, annee_renovation, ancien_nom_ev, annee_changement, presence_cloture, ouvert_ferme, id_atelier_horticolore, competence, last_edited_user, last_edited_date
- id_eqpt -> id_division
- geom_x_y -> geom, plan_url
- geom -> geom_x_y, plan_url
- adresse_numero, adresse_typevoie, adresse_libellevoie -> adresse_codepostal

- Un diagramme entitée association au format utilisé dans le cours (tout autre format sera noté 0). La décomposition précédente et le modèle doivent naturellement correspondre.



- Un script qui crée les tables correspondantes avec un maximum de contraintes d'intégrité.
Voir espaces_verts.sql

- Une requête par jeu de données qui reproduit les données initiales à partir des projections

```
SELECT *, COALESCE(eq.id_division, ev.id_division) AS id_division
FROM espaces_verts ev
LEFT JOIN adresses ad ON ev.adresse_numero = ad.adresse_numero
        AND ev.adresse_typevoie = ad.adresse_typevoie
        AND ev.adresse_libellevoie = ad.adresse_libellevoie
LEFT JOIN equipements eq ON ev.id_eqpt = eq.id_eqpt
LEFT JOIN geo ON ev.geom_x_y = geo.geom_x_y;
```

(Il faudrait sélectionner toutes les colonnes manuellement à la place de * sauf les colonnes id_divisions pour ne pas les dupliquer.)

- Une liste d'anomalies que vous avez détectées dans les données, le cas échéant.
Il y a 8 attributs nsq_espace_vert NULL.

On peut avoir des dépendances fonctionnelles avec nom_ev sur d'autres attributs des espaces verts comme type_ev, mais plusieurs espaces verts peuvent avoir le même nom ce qui est peut être volontaire.
geom et geom_x_y possèdent tous les deux seulement 2 NULL.
Certaines adresses ne sont pas complètes, parfois seul le numéro et le type de voie apparaissent
(Ex: 1 | RUE | | 75010).
id_eqpt a plusieurs 0, parfois des NULLet XXXX qui n'aident pas vraiment à la compréhension. Si cela est pour montrer qu'il n'y a pas d'équipements utilisés on pourrait tout simplement créer une table equipement et ne pas insérer ces données.

Arbres

- Liste des dépendances fonctionnelles non-triviales trouvés.

IDBASE -> ESPECE
IDBASE -> VARIETE OUCULTIVAR
IDBASE -> IDEMPLACEMENT
IDBASE -> HAUTEUR
IDBASE -> CIRCONFERENCE
IDBASE -> STADE DE DEVELOPPEMENT
IDBASE -> REMARQUABLE

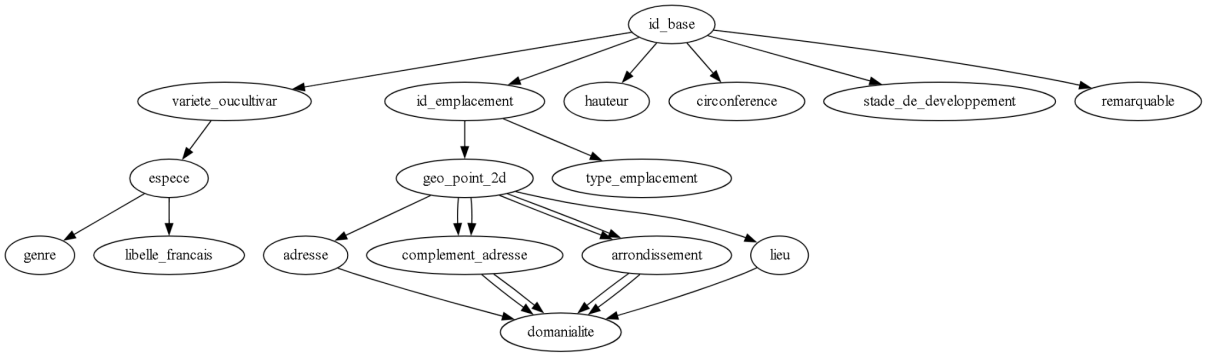
VARIETE OUCULTIVAR -> ESPECE
ESPECE -> GENRE
ESPECE -> LIBELLE FRANCAIS

IDEMPLACEMENT -> geo_point_2d
IDEMPLACEMENT -> TYPE EMPLACEMENT
geo_point_2d -> LIEU / ADRESSE
geo_point_2d -> COMPLEMENT ADRESSE
geo_point_2d -> ARRONDISSEMENT
LIEU / ADRESSE, COMPLEMENT ADRESSE, ARRONDISSEMENT -> DOMANIALITE

- Quelques requêtes que vous développerez pour vérifier que les dépendances fonctionnelles sont effectivement vérifiées dans les données. Concentrez vous sur les cas non-triviaux.

voir arbres_req.sql

- Une représentation graphique d'une couverture minimale de dépendances fonctionnelles. (Vous pouvez utiliser graphviz par exemple.



ou voir abres_graph.png

- Une décomposition en FNBC avec une justification (ne soyez pas exhaustif, concentrez vous sur les cas que vous jugez intéressants et qui méritent une explication).

Genre(genre)
Domaine(domanialite)
Espece(espece, genre, libelle_francais)
Adresse(adresse, complement_adresse, arrondissement, domanialite)
Lieu(adresse, complement_adresse, arrondissement, domanialite)
Variete(variete_oucultivar, espece)
GeoPoint(geo_point_2d, adresse, complement_adresse_adresse, lieu, complement_adresse_lieu, arrondissement)
Emplacement (id_emplacement, geo_point_2d, type_emplacement)
Arbre (id_base, espece, variete_oucultivar, id_emplacement, hauteur, circonference, stade_de_developpement, remarquable)

Les problèmes qui se posaient sur la table sont:

- Problème 1NF: La table d'origine ne respectait pas la 1NF car certains attributs n'étaient pas atomiques (Lieu / Adresse). Nous les avons donc décomposés en 2 attributs distincts pour éviter ce problème.
- Problème 2NF: La 2NF était respecté dû au fait que la relation était identifiée par une clé simple id_base. Cependant, la table contenait de nombreuses redondances à cause des problèmes de dépendances transitives.
- Problème 3NF et FNBC: La 3NF n'était pas respecter dû au dépendances transitives présentes comme id_base -> espèce -> genre ou id_base -> id_emplacement -> adresse. Nous avons décomposé la table pour supprimer ces dépendances transitives en créant de nouvelles tables. Ainsi, les attributs dépendent uniquement d'une clé candidate. Ce qui rend notre table FNBC car dans chaque table, tous les attributs non clés dépendent d'une clé candidate.

Pour les cas GeoPoint et Emplacement, nous avions eu des soucis au début. En effet, id_emplacement et geo_point_2d permettent d'avoir le lieu, l'adresse, le complément d'adresse, l'arrondissement et la domanialité. Donc, nous devions créer 2 tables distinctes pour le lieu etc… c'est pourquoi nous avons choisis de faire ainsi.

- Pour chaque table de la décomposition, l'ensemble des clefs candidates, et les dépendances fonctionnelles locales.

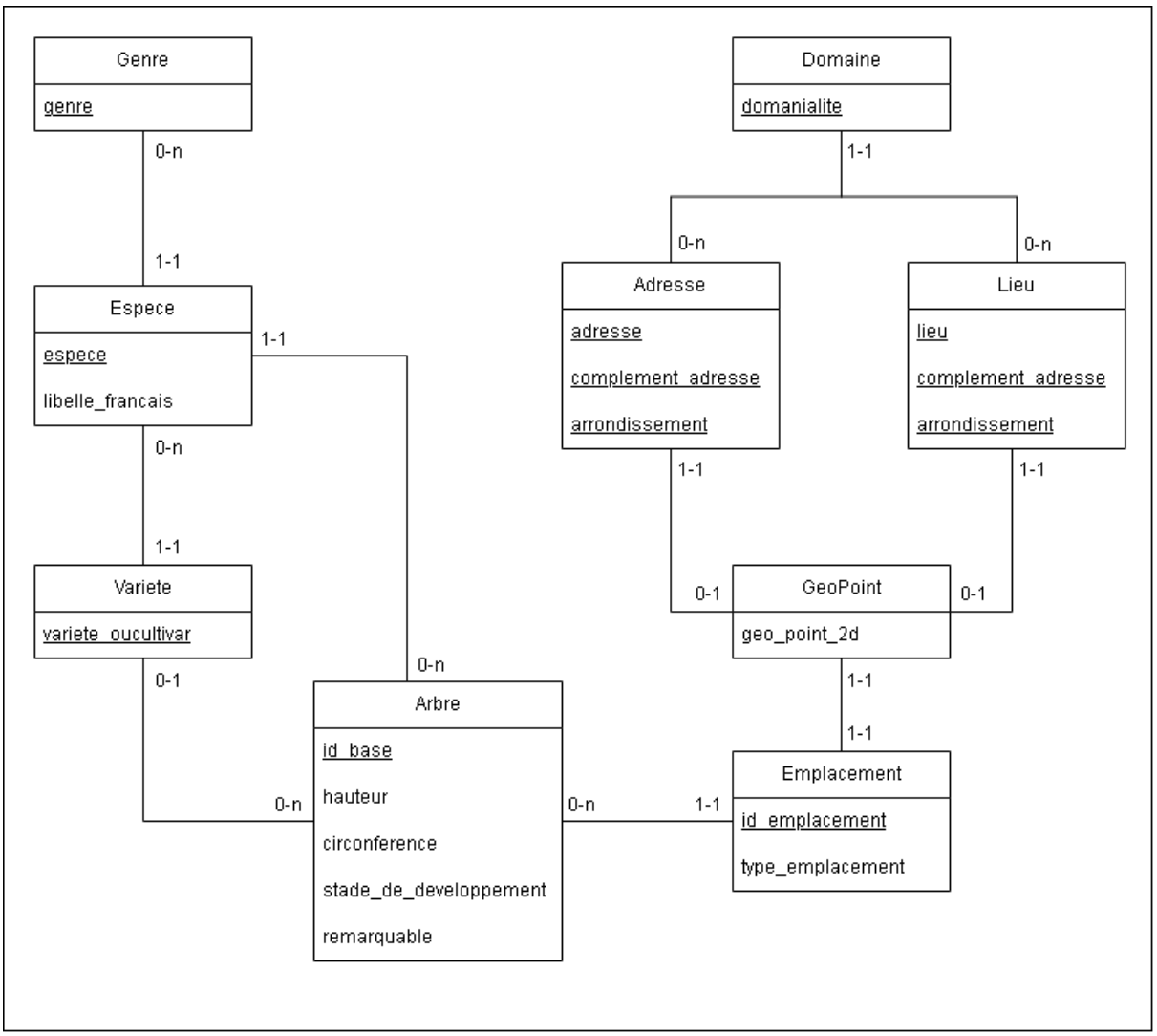
Clé candidates:

- genre
- domanialite
- espece
- adresse, complement_adresse, arrondissement
- lieu, complement_adresse, arrondissement
- variete_oucultivar
- geo_point_2d
- id_emplacement
- id_base

Dépendances fonctionnelles:

- espece -> genre, libelle_francais
- variete_oucultivar -> espece
- adresse, complement_adresse, arrondissement -> domanialite
- lieu, complement_adresse, arrondissement -> domanialite
- geo_point_2d -> adresse, complement_adresse, lieu, arrondissement
- id_emplacement -> geo_point_2d, type_emplacement
- arbre -> espece, variete, id_emplacement, hauteur, circonference, stade_de_developpement, remarquable

- Un diagramme entitée association au format utilisé dans le cours (tout autre format sera noté 0). La décomposition précédente et le modèle doivent naturellement correspondre.



ou voir arbres_er.png

- Un script qui crée les tables correspondantes avec un maximum de contraintes d'intégrité.
voir arbres.sql

- Une requête par jeu de données qui reproduit les données initiales à partir des projections
voir arbres_req.sql

- Une liste d'anomalies que vous avez détectées dans les données, le cas échéant.

Groupe :

CHEN Michel
HUYNH Minh Van Claude